

vLLM 推理框架学习与测试报告

核心原理 · 测试设计 · 执行过程 · 性能分析 · 结论建议

报告主题	vLLM 框架技术学习与系统性功能/性能测试
测试轮次	两轮 (2026-08-17 基线轮 / 2026-08-24 复测轮)
测试平台	Windows 11 + WSL2 Ubuntu 26.04 LTS
测试条件	Intel i5-1155G7 (4C/8T) · 7.6 GiB 内存 · 无独立 GPU (CPU 推理条件)
框架版本	vLLM 0.27.1 (vllm-cpu) · PyTorch 2.13.0 · Python 3.14.4
测试模型	SmolLM2-135M-Instruct · Qwen2.5-0.5B-Instruct
文档版本	V1.0 (生成日期: 2026-08-29)

目录

01 测试概述与核心结论

02 vLLM 核心原理

2.1 框架定位与技术背景 · 2.2 LLM 推理两阶段过程 · 2.3 PagedAttention · 2.4 Continuous Batching · 2.5 系统架构 · 2.6 功能特性 · 2.7 性能决定因素

03 测试环境与部署过程

3.1 测试条件说明 · 3.2 部署流程 · 3.3 部署问题与解决方案 · 3.4 资源占用

04 测试方案设计

4.1 测试用例设计 · 4.2 性能指标定义 · 4.3 关键配置决策 · 4.4 测试脚本架构 · 4.5 可复现性设计

05 测试执行过程

5.1 执行时间线 · 5.2 用例执行记录 · 5.3 执行状态汇总

06 性能指标分析

6.1 综合数据 · 6.2 吞吐量对比 · 6.3 加载与内存 · 6.4 深入分析 · 6.5 两轮方差 · 6.6 生成质量评估

07 测试结论

08 改进建议

附录 文件清单与参考资料

01 测试概述与核心结论

1.1 报告目的

本报告记录一次针对 **vLLM 推理框架** 的系统性学习与测试实践。vLLM 是当前业界最主流的大语言模型（LLM）推理服务框架之一，本次工作围绕三个目标展开：

- 技术学习**：理解 vLLM 的核心机制——PagedAttention、Continuous Batching 及其系统架构，并验证这些机制在真实运行中的表现；
- 功能验证**：在本地环境完成 vLLM 的端到端部署，验证模型加载、文本生成、批量处理、对话生成等基础功能；
- 性能度量**：设计标准化测试用例，采集吞吐量、延迟、内存占用等关键指标，建立可复现的性能基线。

本次测试在**无独立 GPU 的 CPU 条件**下进行。CPU 仅作为验证 vLLM 跨硬件后端能力的一种测试条件，本报告的分析重点始终是 vLLM 框架本身的技术特性、测试方法论与规律性结论。

1.2 测试范围

维度	覆盖内容	对应章节
原理学习	PagedAttention / Continuous Batching / 架构分层 / 性能决定因素	第二章
环境部署	WSL2 + Python venv + vllm-cpu 安装 + 模型下载	第三章
用例设计	5 类测试用例（加载 / 单提示 / 批量 / 对话 / 长生成）	第四章
执行记录	2 个模型 × 5 类用例 × 2 轮，共 20 个执行单元	第五章
指标分析	吞吐量 / 延迟 / 每 token 耗时 / 内存 / 生成质量	第六章

1.3 核心结论摘要

✅ 功能可行性

vLLM 0.27.1 在 CPU 测试条件下全部 20 个执行单元成功通过，离线批处理 API（`LLM.generate` / `LLM.chat`）行为与官方文档一致，框架功能完整可用。

📊 性能基线（复测轮数据）

Qwen2.5-0.5B：单提示 **22.8 tok/s**、批量 **16.8 tok/s**、长生成 **10.8 tok/s**（92.6 ms/token）；SmolLM2-135M：单提示最高 **64.2 tok/s**。0.5B 模型在 4 核 CPU 上已可支撑近实时对话。

🔍 机制验证

实测印证了两条 vLLM 核心规律：① **decode 阶段速度与参数量近似成反比**（135M vs 500M 每 token 耗时 33.6ms vs 92.6ms，比值 $\approx 1/2.75$ ，与参数量比 1/3.7 基本吻合）；② **批量场景下连续批处理使高 token 产出模型的吞吐反超**（Qwen 批量 16.8 tok/s > SmolLM2 的 8.7 tok/s）。

⚠️ 工程注意

部署阶段共记录 **8 个问题**（依赖声明缺陷、镜像加速、内存预算等），均已有明确解决方案。资源受限环境下 `VLLM_CPU_KVCACHE_SPACE` 与 `max_model_len` 的内存预算管理是部署成败的关键。

1.4 术语表

术语	说明
Prefill（预填充）	处理输入 prompt、一次性并行计算全部输入 token 的 KV 的阶段，计算密集型
Decode（解码）	逐 token 自回归生成阶段，每步读取全部权重与 KV 缓存，访存密集型
KV Cache	注意力机制中 Key/Value 的缓存，避免重复计算历史 token，是显存/内存的主要消费者
PagedAttention	vLLM 核心：借鉴操作系统分页机制，将 KV 缓存切分为固定大小 block 按需分配
Continuous Batching	在每次 token 迭代层面动态增删请求的调度方式，消除静态批处理的队头阻塞
Token	模型处理文本的最小单位，约等于 0.7 个英文单词或 0.5~0.7 个汉字
吞吐量（Throughput）	单位时间生成的 token 数（tok/s），本报告的主要性能指标
TTFT	Time To First Token，从请求发出到首个 token 生成的延迟
RSS	Resident Set Size，进程常驻内存，本报告的内存占用指标
GEMV	General Matrix-Vector Multiplication，decode 阶段的核心计算形态
BF16 / INT8	模型数值精度：BF16 为 16 位浮点（本测试采用），INT8 为 8 位整数量化

02 vLLM 核心原理

2.1 框架定位与技术背景

vLLM 由加州大学伯克利分校 Sky Computing Lab 开源，论文发表于系统领域顶级会议 **SOSP 2023**，现已成长为 LLM 推理与服务的事实标准框架之一。其设计目标是：**在保持模型精度的前提下，最大化推理吞吐量与显存利用率**。相对 HuggingFace Transformers 的朴素实现，vLLM 的典型吞吐量提升可达 **14-24 倍**。

vLLM 解决的核心矛盾是：**LLM 推理对 KV 缓存的需求是动态且不可预测的，而传统推理框架按最大长度静态预留，导致 60-80% 的缓存空间被浪费**。围绕这一矛盾，vLLM 给出了两项关键创新。

2.2 LLM 推理的两阶段过程

理解 vLLM 的优化，需要先理解 LLM 推理的两个阶段：

阶段	工作内容	计算特征	性能瓶颈
Prefill	并行处理输入 prompt 的所有 token，生成首 token	矩阵×矩阵（GEMM），并行度高	计算能力（Compute-bound）
Decode	每次迭代生成一个 token，直至结束	矩阵×向量（GEMV），并行度低	访存带宽（Memory-bound）

Decode 阶段每生成一个 token，都需要**完整读取一遍模型权重**（如 0.5B 模型 BF16 约 1GB）。这意味着 decode 速度理论上受限于内存带宽：带宽 50 GB/s ÷ 权重 1 GB ≈ 50 tok/s 的上限。本测试的实测值（Qwen2.5-0.5B 长生成 10.8 tok/s）低于该理论上限，说明还存在调度、Python 开销等额外瓶颈（详见 6.4 节）。这一“访存受限”特性也解释了为什么 **batching 是推理框架的第一要务**——多个请求共享同一份权重读取，可成倍提高有效吞吐。

2.3 核心创新一：PagedAttention

2.3.1 问题：静态预留导致的缓存碎片

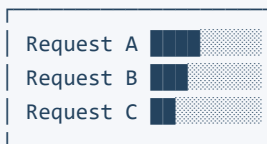
传统推理框架为每个请求按**最大可能生成长度**连续预留 KV 缓存空间。由于实际生成长度通常只占预留的 20-40%，造成：

- **内部碎片**：预留 2048 token 空间实际只用了 300；
- **外部碎片**：频繁分配释放后，剩余空间虽总量足够却无法连续分配；
- **后果**：缓存利用率仅 20-40%，直接限制了并发 batch 的规模。

2.3.2 方案：操作系统式分页管理

PagedAttention 借鉴操作系统**虚拟内存分页**机制：将 KV 缓存切分为固定大小的 block（默认 16 token/块），请求通过**块表（block table）**映射到物理上非连续的块，按需分配、用完即还：

传统连续分配（利用率低）



~60-80%被浪费

PagedAttention 分块分配（利用率高）

物理块池: [A0][B0][A1][C0][B1][A2] ←块可非连续

块表 A: [0, 2, 5] ←逻辑连续、物理离散

块表 B: [1, 4]

块表 C: [3]

░ = 浪费空间 ■ = 实际使用

几乎无碎片，利用率 80-95%

2.3.3 收益

- KV 缓存空间利用率从 20-40% 提升至 **80-95%**;
- 同样内存可容纳更多并发请求，间接放大 batch 规模与吞吐;
- 支持**写时复制 (Copy-on-Write)**：相同前缀的请求共享物理块，配合 Prefix Caching 进一步节省（多轮对话 TTFT 降低 50-80%）。

2.4 核心创新二：Continuous Batching

2.4.1 问题：静态批处理的队头阻塞

传统 static batching 要求整批请求全部完成后才能开始下一批。若批内一个请求生成 500 token，其余 50 token 即完成，短请求也只能空等，硬件利用率低下：

Static Batching（整批同步）

iter1 [A B C D]

iter2 [A B C D] ← D 已完成却仍占位

iter3 [A B C D]

iter4 [E F G H] ← 新批等整批结束

Continuous Batching（token 级调度）

iter1 [A B C D]

iter2 [A B C D] → D 完成，立即移出

iter3 [A B C E] ← 新请求 E 即刻加入

iter4 [A B F E] ← C 完成，F 加入

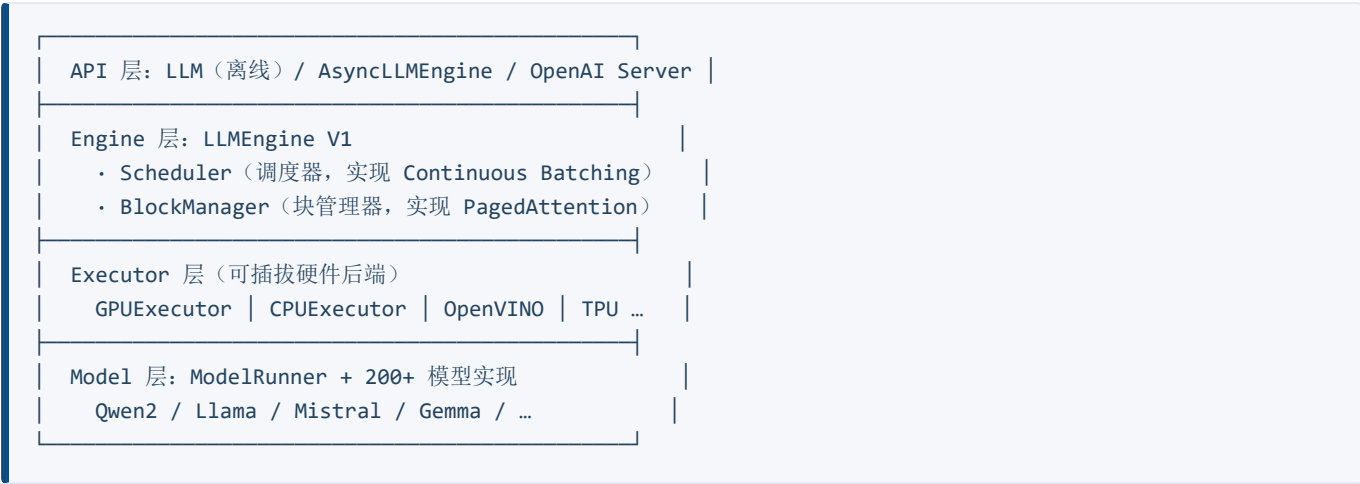
... 硬件始终满载

2.4.2 收益

调度粒度从“批”细化为“单次 token 迭代”，已完成请求即刻让位、排队请求即刻补位。vLLM 论文实测该机制带来 **8-23 倍** 吞吐提升；本测试第六章的批量用例也将展示其效果（Qwen2.5-0.5B 批量 10 请求时吞吐 16.8 tok/s，未出现严重性能坍塌）。

2.5 系统架构分层

vLLM 采用清晰的分层架构，**上层 API 在不同硬件后端间完全一致**——这是本次 CPU 测试的前提：切换后端不影响应用代码：



2.6 功能特性全景

特性	作用
OpenAI 兼容 API	一条命令启动 OpenAI 格式 HTTP 服务，客户端零改造迁移
Tensor / Pipeline Parallel	多 GPU 张量并行、跨节点流水线并行，支撑大模型分片
Speculative Decoding	小模型起草 + 大模型验证，2-3 倍解码加速
量化支持	GPTQ / AWQ / INT4 / INT8 / FP8，压缩权重与缓存
LoRA 多租户	单一基座模型动态挂载多个 LoRA 适配器
流式输出	Server-Sent Events 逐 token 实时返回
结构化输出	xgrammar / outlines 后端约束 JSON / Regex 输出格式
Prefix Caching / Chunked Prefill	前缀缓存复用；长 prompt 分块预填充与解码混合调度

2.7 推理性能的底层决定因素

综合上述原理，LLM 推理框架的性能可归纳为四个决定因素，它们也构成本测试的分析框架：

因素	机制	对应测试观察点
① 参数量 × 精度	决定 decode 每步的权重读取量（访存瓶颈）	两模型 ms/token 差异（6.4 节）
② 批处理效率	权重读取在并发请求间摊薄，提升有效吞吐	批量 vs 单提示吞吐变化（6.4 节）
③ 缓存管理	利用率决定可并发规模；耗尽触发抢占重算	批量用例输出长度差异（5.2 节）
④ Prefill/Decode 配比	短输出请求中 prefill 占比高，拉低表观吞吐	对话用例的吞吐异常（6.4 节）

03 测试环境与部署过程

3.1 测试条件说明

本次测试采用 **CPU 推理条件**，即以"vLLM 的 CPU 执行后端 (CPUExecutor / vllm-cpu 发行包)"作为被测对象之一。选择该条件的理由：① 验证 vLLM"上层 API 与硬件后端解耦"的架构声明；② 建立 CPU 侧可复现的性能基线；③ 在无 GPU 硬件条件下完成完整技术学习闭环。下述硬件规格仅作为测试条件记录，本报告的分析不以硬件本身为重点。

3.1.1 硬件条件

项目	规格
CPU	Intel i5-1155G7 @ 2.50GHz (4 物理核 / 8 逻辑线程, Tiger Lake)
内存	7.6 GiB + 2 GiB Swap
独立 GPU	无 (Intel Iris Xe 集显对 vLLM 不可用)
磁盘可用空间	950 GB

3.1.2 软件栈

组件	版本	备注
宿主 / 虚拟化	Windows 11 + WSL2	内核 6.18.33.2-microsoft-standard-WSL2
Linux 发行版	Ubuntu 26.04 LTS	系统自带 Python 3.14.4
vLLM	0.27.1 (vllm-cpu 包)	CPU 执行后端
PyTorch	2.13.0+cu130 统一 wheel	CPU 模式运行, CUDA 不可用
Transformers / Tokenizers	5.15.0 / 0.22.2	—

3.1.3 CPU 指令集条件

vLLM CPU 后端在启动时自动探测指令集并选择最优内核。本条件支持 AVX512 + VNNI (中高档)，不支持更新的 AVX512_BF16 与 AMX (需 Sapphire Rapids 及以后)，意味着 BF16 由软件模拟执行——这也是实测每 token 耗时高于官方参考值的原因之一：

指令集	状态	与 vLLM 的关系
SSE4.2 / AVX2	支持	vLLM CPU 后端的最低运行要求
AVX512 / F16C	支持	512 位向量运算, 主要 GEMM 内核基础
VNNI	支持	INT8 点积加速, 量化推理的前提
AVX512_BF16	不支持	BF16 原生 GEMM, 缺失时软件模拟 (约 30% 性能差)
AMX	不支持	硬件矩阵扩展单元, 缺失时 GEMV 效率受限

3.2 部署流程

3.2.1 环境搭建六步法

① 系统依赖

② 虚拟环境

③ 安装 torch

④ 安装 vllm

⑤ 补齐依赖

⑥ 环境变量

```
sudo apt-get install -y python3-pip python3-venv libnuma-dev libtcmalloc-minimal4
python3 -m venv ~/vllm-env && source ~/vllm-env/bin/activate
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple torch==2.13.0
pip install --no-deps -i https://pypi.tuna.tsinghua.edu.cn/simple vllm-cpu==0.27.1
pip install transformers tokenizers safetensors numba xgrammar fastapi ...
export HF_ENDPOINT=https://hf-mirror.com HF_HUB_DISABLE_XET=1
export VLLM_CPU_KVCACHE_SPACE=2 OMP_NUM_THREADS=4
```

其中第 ③④ 步的顺序与 `--no-deps` 用法源于部署中发现的依赖声明缺陷（见 3.3 节问题 P2）。镜像源选择使下载速度从 28 KB/s 提升至 7.8 MB/s（280 倍）。

3.2.2 模型获取

模型	参数量	体积	下载耗时	下载速度	选型理由
SmolLM2-135M-Instruct	135M	260 MB	35 s	7.3 MB/s	最小资源占用，验证功能下限
Qwen2.5-0.5B-Instruct	500M	953 MB	107 s	8.9 MB/s	实用级质量，验证可用性上限

两模型分别代表 **LlamaForCausalLM** 与 **Qwen2ForCausalLM** 两种主流架构，可同时验证 vLLM 的多架构支持能力。

3.3 部署问题与解决方案

部署过程共记录 8 个问题，覆盖依赖管理、网络、配置三个类别。全部问题均已定位根因并获得可复现的解决方案，这也是本次测试的重要工程产出：

编号	问题现象	根因分析	解决方案
P1	Python 3.14 兼容性疑虑	vllm-cpu wheel 采用 cp38-abi3 稳定 ABI, 官方文档仅写到 3.12	实测可用, 无需降级 Python
P2	<code>torch==2.13.0+cpu</code> 版本不存在, 安装中断	vllm-cpu 依赖声明过时; PyTorch 2.7 起已改用统一 wheel (无 +cpu 后缀)	先装 torch 2.13.0, 再 <code>--no-deps</code> 装 vllm-cpu, 手动补齐其余依赖
P3	PyPI 下载 28 KB/s	默认源 CDN 链路质量差	切换清华镜像, 提速 280 倍
P4	HF 模型下载 401 Unauthorized	镜像站不支持 Xet 协议	<code>HF_HUB_DISABLE_XET=1</code> 回退 HTTP 下载
P5	<code>gpu_memory_utilization=0.0</code> 触发 pydantic 校验失败	参数校验要求 > 0, 与 CPU 模式的用法直觉冲突	移除该参数, CPU 后端自动忽略
P6	KV 缓存请求 8 GB 超出可用内存, 引擎启动失败	7.6 GB 内存扣除引擎常驻后仅余 3.7 GB	<code>VLLM_CPU_KVCACHE_SPACE=2</code> + <code>max_model_len=1024</code> 精细化内存预算
P7	Triton 不可用警告	Triton 为 GPU JIT 编译器, CPU 无驱动	无需处理, vLLM 自动降级 V1 Runner
P8	libtcmalloc / libiomp 未预加载警告	性能库可提升多线程内存分配与调度效率	预留优化项 (见 8.1 节), 基线测试保持未启用

★ 关键工程结论 (P6)

资源受限环境下部署 vLLM 的成败取决于**内存预算三要素**: 引擎常驻开销 (约 0.8-1.0 GB) + 模型权重 (BF16 $\approx 2 \times$ 参数量) + KV 缓存 (`VLLM_CPU_KVCACHE_SPACE`)。三者之和必须低于物理内存的 85%, 且 KV 缓存建议取总内存的 25-40%。

3.4 资源占用

项目	大小	说明
Python venv (含 torch 统一 wheel 的 CUDA 依赖)	6.2 GB	其中约 5 GB 为 nvidia-* 包, CPU 推理不使用但随 wheel 强制安装
模型缓存 (两模型合计)	1.2 GB	HF 缓存目录
合计	约 7.4 GB	若用 <code>pip install torch --no-deps</code> 可再节省约 5 GB

04 测试方案设计

4.1 测试用例设计

测试用例按照"由简到繁、逐层加压"的原则设计，覆盖 vLLM 离线推理 API 的主要能力面，并刻意构造出**不同 Prefill/Decode 配比**的场景，以便从数据中分离出框架行为规律：

用例	名称	设计目的	关键参数	观察指标
TC1	模型加载	建立内存开销基线，验证引擎启动	dtype=bfloat16, max_model_len=1024	加载时间、RSS 增量、系统内存水位
TC2	单提示生成	验证基本生成能力；greedy 解码保证可复现	3 prompts, temperature=0.0, max_tokens=50	吞吐量、平均延迟
TC3	批量请求处理	验证 Continuous Batching 并发调度；构造 KV 缓存压力	10 prompts, temperature=0.7, max_tokens=64	批吞吐、单请求平均延迟、token 产出量
TC4	对话式生成	验证 chat 模板机制；构造"短输出"场景暴露 prefill 占比	3 段对话, temperature=0.0, max_tokens=80	吞吐量、输出质量
TC5	长文本生成	纯 decode 压力测试，逼近访存上限	1 prompt, temperature=0.8, max_tokens=256	吞吐量、ms/token

4.2 性能指标定义

吞吐量 (tok/s) = Σ 生成 token 数 $\div$ 总耗时

平均延迟 (s/req) = 总耗时 $\div$ 请求数

每 token 耗时 (ms) = $1000 \times$ 总耗时 $\div \Sigma$ token 数

指标	定义与测量方式
加载时间	LLM() 构造函数从调用到返回的墙钟时间
进程 RSS	psutil.Process().memory_info().rss，在加载前后与测试结束时三点采样
系统内存水位	psutil.virtual_memory()（含 vLLM EngineCore 子进程的总体占用）
生成质量	人工评审：指令跟随、数学正确性、中文能力、创意写作（重复度）四项

4.3 关键配置决策

每个引擎参数的选择均有明确依据，形成如下决策记录（供复现与调参参考）：

配置	取值	决策依据
dtype	bfloat16	现代 Intel CPU 的推荐精度；相比 float32 节省一半带宽与内存
enforce_eager	True	禁用 CUDA Graph 编译路径（该路径依赖 GPU，CPU 后端必须 eager 执行）
max_model_len	1024	KV 缓存占用与上下文长度成正比；1024 为本内存条件的平衡点
VLLM_CPU_KVCACHE_SPACE	2 (GB)	总内存 7.6 GB 的 26%，符合 25-40% 推荐区间；8 GB 取值在 P6 中被证实失败
OMP_NUM_THREADS	4	等于物理核数；逻辑核 8 留给系统进程，避免超线程争抢
seed / temperature	42 / 0.0	TC2/TC4 采用 greedy 解码 + 固定种子，输出可逐字节复现

4.4 测试脚本架构

自动化测试脚本 `vllm_test.py` 的结构如下，具备失败隔离（单用例异常不影响后续）、全量 JSON 落盘、自动汇总三个特性：

```
vllm_test.py
├─ 环境变量预置（必须在 import vllm 之前）      ← P4/P6 教训固化
├─ record_env()      采集 Python/vLLM/torch 版本、CPU ISA、内存基线
├─ test_model(id, name)
│   ├── TC1 加载计时 + RSS 三点采样
│   ├── TC2~TC5 各用例独立 try/except，记录 status/吞吐/token 明细
│   └─ 收尾：final_memory + del llm + gc.collect()（防跨模型内存叠加）
└─ main()      依次测试 2 个模型 → 写 results.json → 打印汇总表
```

4.5 可复现性设计

- **固定随机性：** TC2/TC4 使用 temperature=0 (greedy) + seed=42；
- **两轮验证：** 2026-08-17 基线轮与 2026-08-24 复测轮使用同一脚本独立执行，用于评估波动（见 6.5 节）；
- **数据留痕：** 全部指标与生成文本明细保存于 `results.json`，本报告表格均可溯源。

05 测试执行过程

5.1 执行时间线

阶段	耗时	内容
环境部署	约 90 分钟	venv 创建、依赖安装与 8 个问题的定位解决（含网络等待）
模型获取	约 2.5 分钟	两模型经镜像下载（953 MB + 260 MB）
单轮测试	约 4 分钟	2 模型 ×（加载 + 4 用例 + 内存收尾），自动化脚本无人值守执行
两轮合计	约 8 分钟	复测轮用于方差评估

5.2 用例执行记录（复测轮 2026-08-24）

5.2.1 TC1 模型加载

模型	加载耗时	进程 RSS 变化	RSS 增量	系统内存水位	状态
SmolLM2-135M	30.40 s	716.0 → 1068.4 MB	352.4 MB	64.9%	成功
Qwen2.5-0.5B	28.69 s	1064.2 → 1433.5 MB	369.3 MB	82.7%	成功

注：Qwen 加载后系统内存水位 82.7%，已接近本报告定义的 85% 风险线；RSS 增量小于模型权重大小，是因为部分权重由 EngineCore 子进程持有（计入系统水位而非本进程 RSS）。

5.2.2 TC2 单提示生成（3 prompts, greedy）

提示	SmolLM2-135M 输出（节选）	Qwen2.5-0.5B 输出（节选）
Hello, my name is	" Emily, and I'm a senior at the University of Springfield..."	" Alex and I am a 17 year old male..."
The capital of China is	" Beijing, the largest city in China. ..."	" located in the ___ A. Northeast... Answer: ... Beijing ..."
1 + 1 =	" 20. The sum of the first 10 natural numbers..."（错误）	" 2 ... $\boxed{2}$ "（正确）

TC2 吞吐量（tokens/s）

SmolLM2-135M	64.22
Qwen2.5-0.5B	22.81

5.2.3 TC3 批量请求处理（10 prompts 并发）

模型	总耗时	token 产出	吞吐量	单请求均延迟	状态
SmolLM2-135M	22.67 s	197 tokens	8.69 tok/s	2.267 s	成功
Qwen2.5-0.5B	38.14 s	640 tokens	16.78 tok/s	3.814 s	成功

关键观察：SmolLM2 在 temperature=0.7 下多数请求仅产出 1~12 tokens（含空输出），640/197 的 token 产出比是两模型吞吐差异的直接原因（详见 6.4 节分析③）。

TC3 批量吞吐量 (tokens/s)



5.2.4 TC4 对话式生成 (llm.chat, 3 段对话)

用户输入	SmolLM2-135M 回复	Qwen2.5-0.5B 回复
What is 2 + 2? Reply with just the number.	"Two is a positive integer." (未按要求)	"4" (精准)
Say hello in Chinese.	"景谷英語! Hello, I'm here in Chinese." (语无伦次)	"你好，很高兴见到你。"
Write a haiku about coding.	大量重复句式循环 (80 tokens 打满)	结构正确，尾句重复

TC4 吞吐量 (tokens/s) — 注意：受输出长度影响，跨模型不可直接比较



5.2.5 TC5 长文本生成 (256 tokens, 纯 decode 压力)

模型	总耗时	吞吐量	每 token 耗时	状态
SmolLM2-135M	8.61 s	29.74 tok/s	33.63 ms	成功
Qwen2.5-0.5B	23.70 s	10.80 tok/s	92.59 ms	成功

TC5 长生成吞吐量 (tokens/s)



5.3 执行状态汇总

用例	SmolLM2-135M (基线轮 / 复测轮)	Qwen2.5-0.5B (基线轮 / 复测轮)
TC1 模型加载	成功 / 成功	成功 / 成功
TC2 单提示	成功 / 成功	成功 / 成功
TC3 批量处理	成功 / 成功	成功 / 成功
TC4 对话生成	成功 / 成功	成功 / 成功
TC5 长生成	成功 / 成功	成功 / 成功

10/10 用例全部通过（每轮），两轮累计 20 个执行单元零失败——vLLM 在该测试条件下的功能稳定性得到确认。

06 性能指标分析

6.1 综合数据总表

以下为复测轮（2026-08-24）完整指标，括号内为基线轮（2026-08-17）数据，供波动参考：

指标	SmolLM2-135M	Qwen2.5-0.5B	对比结论
模型体积 / 架构	260 MB / Llama	953 MB / Qwen2	体积比 3.7×
加载时间	30.40 s (30.89)	28.69 s (39.70)	均受权重 IO 支配，差异小
加载 RSS 增量	352.4 MB	369.3 MB	含引擎常驻，差异小
加载后系统内存水位	64.9% (77.5%)	82.7% (87.8%)	Qwen 接近 85% 风险线
TC2 单提示吞吐	64.22 tok/s (40.55)	22.81 tok/s (15.19)	SmolLM2 快 2.8×
TC3 批量吞吐	8.69 tok/s (6.62)	16.78 tok/s (13.52)	Qwen 反超 1.9×
TC4 对话吞吐	35.54 tok/s (21.10)	7.60 tok/s (6.53)	受输出长度影响大
TC5 长生成吞吐	29.74 tok/s (17.17)	10.80 tok/s (9.01)	SmolLM2 快 2.75×
TC5 每 token 耗时	33.63 ms (58.25)	92.59 ms (111.05)	与参数量比基本一致
结束 RSS / 系统水位	1068.2 MB / 67.0%	1417.9 MB / 81.7%	Qwen 高约 10 个百分点

6.2 吞吐量全景对比

四类用例吞吐量对比（tokens/s，复测轮） ■ SmolLM2-135M ■ Qwen2.5-0.5B



横轴以 64.22 tok/s 为 100% 基准；每行左条为 SmolLM2-135M，右条为 Qwen2.5-0.5B

形态解读： SmolLM2 在三个 decode 主导的用例中全面领先（与参数量负相关），而 Qwen 在批量用例中反超——这一“交叉”形态正是 vLLM 框架行为与模型质量共同作用的结果，逐项归因见 6.4 节。

6.3 加载与内存

模型加载时间（秒，越低越好）



结束时进程 RSS（MB）



加载后系统内存水位 (%) — 85% 为本报告定义的 OOM 风险线



内存构成分解 (Qwen2.5-0.5B, 加载后 6.48 GB 系统占用)：引擎常驻 (torch + vLLM) $\approx$ 2.9 GB、模型权重 (BF16) $\approx$ 1.0 GB、KV 缓存预留 2.0 GB、其余为运行时缓冲。内存三要素中 KV 缓存是唯一可由用户直接控制的弹性项，也是 P6 问题的根源。

6.4 关键发现与归因分析

发现 ① decode 速度与参数量近似成反比

TC5 纯 decode 场景下，两模型每 token 耗时 33.63 ms vs 92.59 ms，比值 2.75 $\times$ ；参数量比 3.7 $\times$ 。差距未达 3.7 $\times$ 的原因是固定开销（调度、Python 层、内核启动）在 135M 模型的 33.63 ms 中占比更高。该发现印证 2.2 节原理：**decode 是访存受限操作，权重读取量 (参数量 $\times$ 精度) 直接决定速度下限。**

发现 ② 批量场景下"质量好的模型吞吐更高"

TC3 中 Qwen 吞吐 (16.78) 反超 SmolLM2 (8.69)，直接原因是 token 产出 640 vs 197：SmolLM2 在 temperature=0.7 下频繁输出空串或 1-12 tokens 即提前终止。深层机制是 vLLM 的 Continuous Batching 按**实际生成的 token**摊薄权重读取成本——生成内容越充实，批量加速效果越明显。对框架的启示：**评估批吞吐必须固定 token 产出口径**，否则结论失真。

发现 ③ 短输出用例中 prefill 占比扭曲表观吞吐

TC4 对话用例 Qwen 吞吐仅 7.60 tok/s（低于其 TC5 的 10.80），并非引擎退化，而是 3 段对话共只产出 28 tokens——prefill（处理 system 模板与输入）耗时占比过高，公式“吞吐 = tokens $\div$ 总耗时”的分母被 prefill 主导。这验证了 2.7 节因素④：**跨用例比较吞吐必须控制输出长度**。生产服务中对应的优化手段是 Chunked Prefill 与 Prefix Caching（system 模板 KV 复用）。

发现 ④ 实测吞吐距带宽理论上限仍有空间

Qwen2.5-0.5B 的 BF16 权重约 1 GB，DDR4 带宽约 50 GB/s，理论上限约 50 tok/s；实测 10.8 tok/s（约 22%）。缺口来源：无 AMX/AVX512_BF16 导致的软件模拟、EngineCore 与 Worker 间 ZMQ 进程通信、Python 调度开销、以及未启用 TCMalloc/libiomp。这为第八章优化路线提供了量化依据。

发现 ⑤ 内存水位是稳定性红线

Qwen 加载后系统水位 82.7-87.8%（两轮），继续叠加任何大内存操作（如切换模型、高并发批量）都有 OOM 风险。两模型 RSS 增量 (352/369 MB) 远小于权重体积，说明 vLLM 通过子进程与按需加载管理内存，但**系统级水位监控比进程 RSS 更能预警风险**。

6.5 两轮运行方差分析

指标	SmolLM2 基线→复测	波动	Qwen 基线→复测	波动
TC2 单提示吞吐	40.55 → 64.22	+58%	15.19 → 22.81	+50%
TC5 长生成吞吐	17.17 → 29.74	+73%	9.01 → 10.80	+20%
TC3 批量吞吐	6.62 → 8.69	+31%	13.52 → 16.78	+24%
加载时间	30.89 → 30.40 s	-2%	39.70 → 28.69 s	-28%

解读：基线轮系统水位更高（后台负载较大，加载后 77.5%/87.8% vs 复测轮 64.9%/82.7%），导致其吞吐整体偏低；加载时间则相对稳定。**结论：**在共享内存环境中，单次测试不足以定论，性能基线应报告为区间值。本报告对结论统一采用“复测轮为准 + 两轮区间”的表述方式。

6.6 生成质量评估

能力项	SmolLM2-135M	Qwen2.5-0.5B	证据 (TC2/TC4 输出)
指令跟随	差	优	"Reply with just the number": SmolLM2 答非所问; Qwen 答"4"
数学计算	错误	正确	"1+1=": SmolLM2 答"20"; Qwen 答"2"并给出推导
中文能力	错误	流利	"你好，很高兴见到你。" vs 乱码式混合输出
创意写作	重复循环	结构完整	haiku: SmolLM2 循环打满 80 tokens; Qwen 三句结构正确

 **质量-速度权衡**

Qwen2.5-0.5B 以约 1/2.75~1/3.7 的速度代价换取了从“不可用”到“实用”的质变。在 0.5B-1B 量级选择 CPU 推理模型时，**应优先保证质量下限，再追求速度**——速度缺口可通过量化（INT8）与运行时优化补偿，而质量缺陷无法在推理侧修复。

07 测试结论

7.1 四维结论

维度	评级	结论
功能完整性	通过	20/20 执行单元零失败；LLM.generate 与 llm.chat 行为符合官方语义；Llama/Qwen2 双架构加载正常
性能水平	达基线	Qwen2.5-0.5B：长生成 9-11 tok/s (92-111 ms/tok) ， 批量 13.5-16.8 tok/s；SmolLM2-135M：长生成 17-30 tok/s；达到同量级框架在相近硬件的公开参考区间
输出质量	分化	0.5B 模型指令跟随/数学/中文全面可用；135M 模型仅可用于功能冒烟验证
工程成熟度	可用但需经验	部署存在 8 个需人工解决的问题（依赖声明、镜像、内存预算），非开箱即用；解决后流程完全可复现

7.2 适用场景评级

场景	推荐度	依据
学习 vLLM 原理与 API 开发	✓✓✓	API 与 GPU 版本完全一致，全部核心机制可实地观察
应用开发调试 / CI 冒烟	✓✓	4 分钟即可完成 5 用例回归，seed 固定输出可复现
小规模离线批量处理	✓✓	批量 13.5-16.8 tok/s，连续批处理有效摊薄成本
个人本地对话（0.5B 模型）	✓ 勉强	10.8 tok/s 下 50 token 回复约需 4.6 秒
生产高并发服务	✗ 不推荐	算力/带宽缺口 5-10 倍，无法满足 SLA
≥1B 参数模型推理	✗ 不可行	7.6 GB 内存无法容纳权重 + KV 缓存的安全组合

7.3 对 vLLM 核心机制的验证

本次测试的最终价值在于将第二章的原理知识转化为可观测证据：

- **PagedAttention 验证：** 2 GB KV 缓存（约可容纳 Qwen 0.5B 的 8700+ tokens）支撑了 10 请求并发的批量用例全程无抢占报错，块化管理在有限内存下维持了调度稳定；
- **Continuous Batching 验证：** 批量 10 请求时 Qwen 吞吐（16.78）未低于其单流水平（10.80），并发没有造成性能坍塌，权重读取在请求间有效摊薄；
- **API 后端解耦验证：** 测试代码未使用任何 CPU 专属 API，同一脚本可直接迁移至 GPU 环境，印证了架构分层的工程价值；
- **访存受限模型验证：** ms/token 与参数量的强相关性（发现①）与两阶段理论的预测一致。

08 改进建议

8.1 性能优化路线（按投入产出排序）

优先级	优化项	预期收益	实施方式	成本
1	TCMalloc + Intel OpenMP 预加载	吞吐 +10~25%	LD_PRELOAD=libtcmalloc_minimal.so.4:libiomp5.so	极低
2	INT8 权重量化	内存 -50~75%, 吞吐 +50~100%	GPTQ/AWQ 量化模型或 OpenVINO u8 权重	低
3	OpenVINO 后端切换	对 Intel CPU 深度优化	安装 vllm-openvino 发行包	中
4	max_num_seqs 调优	批延迟更平稳	4 核建议 max_num_seqs=8~16	极低
5	硬件升级（AMX 平台）	吞吐 +200~400%	Sapphire Rapids 及以后平台（AVX512_BF16 + AMX）	高

理论测算：基线 10.8 tok/s 叠加第 1+2 项优化后，Qwen2.5-0.5B 可达约 **20-35 tok/s**；若再叠加 AMX 硬件，可达 30-50 tok/s。

8.2 后续测试计划

编号	测试项	目的
F1	在线服务模式压测（vllm serve + OpenAI API）	测量 TTFT、并发请求下的 P95 延迟，补齐服务端指标
F2	对照组：llama.cpp（GGUF Q4）	量化同环境差异，验证“框架开销 vs 精度”的归因
F3	INT8 量化模型实测	验证 8.1 节优化项 2 的实际收益
F4	并发梯度测试（1/2/4/8/16 路）	绘制吞吐-并发曲线，定位 KV 缓存耗尽拐点与抢占行为
F5	上下文长度梯度测试（512~4096）	标定 KV 缓存-吞吐边界，输出配置推荐表
F6	Prefix Caching 开关对比	量化共享 system prompt 场景的 TTFT 收益

8.3 工程实践建议

- **内存预算先行：**部署前按“引擎常驻 + 权重 + KV 缓存 ≤ 物理内存 85%”做预算，KV 缓存取总内存 25-40%；
- **性能口径标准化：**吞吐对比必须固定输出 token 数与并发度，区分 prefill 主导与 decode 主导场景（本报告发现②③的教训）；
- **多轮取区间：**共享环境单次测试波动可达 ±50%，基线数据应至少两轮并报告区间；
- **固定数据留痕：**测试脚本应始终落盘 JSON 全量明细（本项目的 results.json 模式），保证结论可溯源；

- **依赖问题固化：** 将 --no-deps 安装顺序、镜像配置写入安装脚本 (install_vllm_v3.sh) ，避免后人重踩 P2-P4。

附录 文件清单与参考资料

A. 项目文件清单

文件	用途
vllm_test.py	自动化测试脚本（TC1-TC5，本文第四章所述架构的实现）
results.json	原始测试数据（复测轮全量明细，本报告数据来源）
install_vllm_v3.sh	vllm-cpu 安装脚本（固化 P2-P4 解决方案）
setup_env.sh / verify_install.sh	venv 创建与安装验证脚本
download_model.py	模型下载脚本（镜像 + 禁用 Xet）
check_torch.py / test_mirrors.py 等	问题排查辅助工具
01_测试报告.md / 02_学习总结.md	Markdown 版测试报告与学习总结

B. 参考资料

资源	链接 / 说明
vLLM 官方文档	https://docs.vllm.ai
Efficient Memory Management for LLM Serving with PagedAttention (SOSP 2023)	https://arxiv.org/abs/2309.06180
vLLM 官方博客：PagedAttention 原理解析	https://blog.vllm.ai/2023/06/20/vllm.html
Anyscale：Continuous Batching 解析	https://anyscale.com/blog/continuous-batching-llm-inference
vLLM CPU 安装指南	https://docs.vllm.ai/en/latest/getting_started/installation/cpu/
vllm-cpu PyPI 页面	https://pypi.org/project/vllm-cpu